Totality — Founding Brief

A WeChat-style super app: native, integrated versions of the owner's essentials in one app.

Prepared for upload into Claude Code · October 28, 2026 · Owner: McKay Clemens

How Claude Code should use this document: treat it as the founding specification. Do not start writing feature code until the foundation architecture and data model in Sections 5–6 have been proposed, reviewed, and approved. Build one module at a time, in the order in Section 7. Enforce the two-user test in Section 6 for every module. Federation and end-to-end encryption (Section 14) and the AI section (Section 15) are locked principles, not optional extras. Where a decision is marked OPEN in Section 17, ask — do not guess. ***Version 4 note (owner's revision, Oct 28): v1 is a NATIVE iPhone app with a connected Mac app — not a web app. Group messaging is Discord-like. Shopping is Amazon-LIKE, not an Amazon wrapper. Stages A and B must not require spending money. Self-hosting on the owner's Raspberry Pi is the planned hosting direction. Version 5 note (Oct 28): the product is named Totality, and the identity basis is DECIDED — handle + email, phone optional (Section 17). Version 6 note (Oct 28): Docs is folded into the Reader — documents are a content type inside the Reader module; there is no separate Docs module.***

1. Vision

Build **one app containing the owner's own native versions** of the services he uses every day — not a dashboard, launcher, or control center that links out to other apps. One account, one identity, one design system, one search, one notification system, one storage layer. The modules are his services, built by him, in the tradition of WeChat: WeChat did not integrate other companies' apps; it built its own messaging, then its own payments, then everything else on top of its own network.

Core essentials (from the owner's actual usage): Messages, Calling, one unified social media app (text posts, photos, and short video as formats inside a single social module — not three separate modules, with a Lemon8-style discovery homepage), a Drive/Obsidian-style **Reader** for files, notes, and documents — Docs folded into the Reader, not a separate module — with Speechify-style listening as a **Listen button/function inside the Reader**, not a separate module or section — Gmail-style email, Rocket Money-style finance, YouTube Music-style music, Calendar, a dedicated **AI section** powered by an open-source/open-weights model (Qwen family by default) acting as the owner's personal assistant in the role Muse plays for him now — plus the three categories a first pass missed: **shopping** (Amazon-style), **photos**, and **payments** (Venmo-style). Supporting services: browser (Safari-style), clock, news, maps, Canvas (school), banking, fitness, AllTrails-style outdoors, Hevy-style workout tracking, HotSchedules-style work shifts, Discord-style team chat.

Locked cross-cutting principle (owner's decision): everything is federated and end-to-end encrypted wherever the underlying activity permits it. No module is a closed silo by default, and no private content is readable by the server by default. The exact protocol choices and the named exceptions (public posts, ordinary email, regulated payments, bank feeds) are specified in Section 14 — read it before proposing the architecture.

2. Scope Decision (locked)

Build A, designed so B can join at any point — even immediately — with C as the long-term goal.

- **A — Personal:** built for the owner first; he is the first and initially only active user.

- **B — Inner circle:** family, friends, climbing team, reading group can sign up and join at any time with no rebuild or data migration. B-readiness is an architectural requirement from day one (Section 6), not a later phase.
- **C — Public super app:** the long-term goal. Design for C (managed, scalable infrastructure; moderation and safety hooks) but do not build C-stage infrastructure, operations, or licensing now.
- **Budget rule for A and B (owner's decision):** stages A and B must not require spending money — free tiers, open-source software, and hardware the owner already has or plans to own (Section 12). Anything that would force a paid service before C is a design failure; find the free path or defer the feature.

3. Function vs. Network — the Governing Distinction

Building a *function* is almost always doable. Replicating a *network* is the hard part. Every module falls into one of three classes:

Class 1 — Native version buildable, useful from day one

- **Calendar** — own calendar; may sync Google Calendar underneath.
- **Email** — own mail client; email is an open protocol (Gmail API / IMAP underneath, native UI on top).
- **Photos** — own photo cloud (object storage, thumbnails, albums).
- **Reader (files / notes / documents) with Listen built in** — one Drive/Obsidian-style module for files, notes, and documents. **Docs is folded into the Reader (owner's decision):** there is no separate Docs module — documents are a content type inside the Reader, written and edited in the Reader's own editor (TipTap + Yjs) and filed alongside files and notes. Speechify is NOT a separate module: every readable item in the Reader (PDF, ebook, doc, note, saved article) has a **Listen button** that turns it into audio in place — play/pause, speed, voice, queue, and resume position live inside the Reader and the shared media player. His Speechify library (being mirrored to Google Drive as “Speechify Library”) is simply the Reader's first imported collection.
- **Music / audio** — own player over the owner's own library: files, podcasts via RSS, and Reader listening (above) through the same playback engine.
- **AI assistant** — a dedicated AI section (Section 15) on an open-weights model (Qwen by default, swappable). Useful solo from day one over the owner's own Reader files, notes, calendar, and money view, under explicit per-module permissions.
- **Money (view)** — own Rocket Money: Plaid feeds native balances, transactions, and subscription auditing.
- **Maps, browser, clock, news, fitness** — MapLibre/Google Maps SDK, WebView browser, RSS/news APIs, HealthKit in a later native version.

Class 2 — Buildable, but starts empty (the network is the product)

- **Messages — 1:1 Snapchat-shaped and decluttered; group messaging Discord-like (owner's decisions).** Build on a proven federated, end-to-end encrypted foundation — Matrix (Olm/Megolm, MLS-track) is the default candidate, per Section 14; never write a chat protocol from scratch. **Direct messages** use Snapchat's core without its clutter: a clean friends/chat list, fast visual messaging (photo and short video snaps as first-class messages), camera one swipe away, close-friends emphasis, and optional disappearing messages — chosen per conversation, never forced. Explicitly OUT: no Discover tab, no Spotlight tab, no ads, no promoted stories, no map-by-default, no streak

anxiety mechanics as dark patterns (a simple streak counter is allowed only if the owner asks). **Groups, by contrast, are modelled on Discord:** communities/spaces that contain named topic channels, threads inside channels, and light roles/permissions — the shape his real groups (climbing team, reading group) already organise in. DMs stay personal and minimal; groups get structure. Value depends on who else joins — hence the B-ready, federated foundation.

- **Calling** — in-app voice/video between users via WebRTC (LiveKit), end-to-end encrypted for 1:1 calls. Calling real phone numbers is separate: PSTN provider (e.g. Twilio), per-minute cost, caller-ID rules, and PSTN calls by definition cannot be end-to-end encrypted — label them as such in the UI.
- **Social — ONE unified module with a Lemon8-style homepage (owner's decision).** X, Instagram, and TikTok are not three modules. There is one social app: one profile, one follow graph, one composer, one comments/thread system, one notification stream. What those platforms split into separate apps is here just *content format*: a text post, a photo post, and a vertical short video are three formats a post can take — and a single post can mix them. **The homepage is Lemon8-style:** a discovery-first, two-column masonry/card feed, topic/category chips and strong search at the top, aesthetic cover cards (image, title, author, saves/likes) regardless of the post's format — a text post gets a designed text cover card, a video post a thumbnail with a play badge. Filter chips may narrow the homepage by format or topic (All / Text / Photos / Video / topic) without splitting the module. The social module's direct messages are not a separate inbox: they are the Messages module, reached from inside social (a module contract, Section 6). This is the owner's *own* federated social network (Section 14). Importing Instagram/TikTok/X content or users is impossible: no API, no licence, and it would violate those platforms' terms. Reference implementations: Bluesky for the unified post record, Lemon8 for the homepage's look and discovery logic (Section 13).

Class 3 — Cannot be done by coding alone (regulated)

- **Payments** — moving money is regulated; there is no money-transmitter licence in this plan. Build the wallet UI and ledger on partner rails (Stripe / Dwolla / banking-as-a-service); the partner holds the actual accounts.
- **Shopping — Amazon-LIKE, not an Amazon wrapper (owner's decision).** The shopping module is its own marketplace modelled on Amazon's experience — rich product pages, powerful search and filtering, recommendations, cart, one-flow checkout, order tracking — but it is NOT a front-end, wrapper, or reskin of Amazon: Amazon does not license that, and the catalogue, data, and checkout are the app's own (own/supplier/affiliate catalogues, Stripe checkout). Amazon is the design and UX benchmark, never the backend.

4. Hard Platform Constraints (do not fight these)

- iOS will not allow an app to replace the OS, read iMessage, intercept ordinary phone calls, operate as an Apple Pay wallet, or absorb TikTok/Instagram feeds. Design within this from the start.
- **v1 is a native iPhone app (owner's decision, v4 — this supersedes the earlier web-app plan).** The first version must be an app the owner can install and use on his own iPhone. It does NOT need to be publicly available or App Store listed at v1: personal distribution (direct install / TestFlight-class) is acceptable. Build in Swift/SwiftUI, which also unlocks the native-only features a super app needs: PhotoKit, widgets, lock screen, CallKit for calls, and the share sheet.
- **A Mac app is part of the product, not an afterthought:** a macOS companion built from the same SwiftUI codebase and the same account, connected and synced with the iPhone app (same identity,

same data, same E2E keys via the Section 14 key design). Messages, Reader, Calendar, AI section, and money view must work on the Mac; iPhone-first features (camera snaps) may be iPhone-only.

- **iPhone Duo support is a later, explicit requirement:** the owner plans to move to Apple's Duo-class device; when it ships, the app must adapt to its form factor (fold/dual-pane layouts, continuity between folded and unfolded states) without a rewrite. Design layout code adaptively from the start (size classes, no hard-coded single-screen assumptions) so Duo support is a layout project, not a rebuild.

5. Architecture — One Modular Monolith

Native clients, one backend. The app is a SwiftUI multiplatform codebase (iPhone + Mac) with a **single** Supabase backend. Each essential is a module inside the client monolith sharing the foundation layer. Do not build twelve microservices; a solo builder drowns in that. Keep services open-source/free-tier and self-hostable so the whole backend can later move onto the owner's own server (below) — growth to C must be a matter of configuration and hosting, not re-architecture.

Layer	Choice	Purpose
App framework	SwiftUI multiplatform (iOS + macOS), Swift	One native codebase for iPhone and Mac; adaptive layout from the start so iPhone Duo support is not a rewrite
UI	Native SwiftUI design system on project design tokens (Section 16)	One look across every module and both platforms; web component registries now serve as pattern references, not the component base
Backend	Supabase (Auth, Postgres, Storage, Realtime) — free tier for A/B; self-hostable	Identity, data, files, and messaging/presence in one service that can later move wholesale onto the owner's server
Self-hosting (planned)	Owner's Raspberry Pi home server	Eventual home for the backend, the Matrix/federation homeserver, media storage, and possibly local Qwen inference (Sections 14–15). Reached securely (Tailscale-class private networking or equivalent). The Pi is the C-independent hosting direction; cloud free tiers are the on-ramp, not the destination
Calls	LiveKit (cloud)	WebRTC voice/video without running media servers
Documents	TipTap + Yjs	Rich text editing and real-time collaboration
Money data	Plaid SDK	Balances, transactions, subscriptions (read-only)
Google services	googleapis (Gmail, Calendar, Drive)	Email/calendar/files sync where the owner wants it
Email sending	Resend	Invites and transactional email
Validation	Zod	One schema language at every module boundary
Testing	XCTest/Swift Testing + simulator & device verification; Playwright only for any web surfaces/APIs	Unit tests plus on-device verification of every module on iPhone and Mac

Layer	Choice	Purpose
Federation	Open protocols: Matrix (messages/calls signalling), ActivityPub and/or AT Protocol (social) — final pick in Phase 0 proposal	Users on other servers/apps can interoperate; identity and graph are portable, not locked to one server (Section 14)
Encryption	Client-side E2E by default: MLS / Double Ratchet-class protocols for messages & calls; user-key encryption (libsodium/age-class) for private files, docs, photos, AI memory	Server stores and relays ciphertext for private content; keys live on user devices (Section 14)
AI model	Qwen open-weights family by default, behind an OpenAI-compatible, model-swappable inference layer (self-hosted via vLLM/Ollama-class serving, or a chosen hosted provider)	The AI section (Section 15) is not hard-wired to one vendor; the owner can change model/provider without rebuilding the app
Later modules	Stripe (commerce), Sentry (errors), MapLibre (maps)	Added when their module arrives, not before

Architecture tension Claude Code must resolve in the Phase 0 proposal, not ignore: Supabase is the pragmatic solo-data backend, but federation + E2E push identity, messaging, and social toward protocol servers (Matrix / ActivityPub / AT Protocol) and client-held keys. Propose the seam explicitly — what lives in Supabase, what lives in protocol servers, and how one identity spans both — and get it approved before building.

6. Foundation — the “OS Layer” (build first, non-negotiable)

- **Multi-user data model from the first table.** Every record (note, photo, message, transaction) has an owner_id; every query is permission-checked; Supabase row-level security is ON from day one.
- **Real identity:** handles/usernames, profiles, a contacts system, and an invite flow are foundation features, not a later phase. A B user who joins must be able to sign up, find the owner by handle, and be inside the app immediately.
- **Visibility on everything:** every post, photo, and document is created with a visibility state — private / contacts / public — even while only “private” is used. Retrofitting this later is extremely painful.
- **Shared services:** universal search across all modules, one notification system, one storage layer, one audit-log table.
- **Safety hooks stubbed in:** block and report functions in messaging/social, account deletion, and data export (required by app stores anyway). Terms/privacy pages and an age gate precede any public (C) launch.
- **Module contracts are the product:** what makes this an everything app is interconnection — e.g. a message can attach a document, a photo, or a payment; a calendar event can attach a doc and a call; the Reader's Listen button feeds the shared player; the AI section can (with permission) search the Reader, calendar, and money view. These contracts are decided deliberately by the owner and

recorded before modules are built.

- **Federated identity from day one:** a portable identity (handle in the form @user@home-server/domain, backed by the federation protocols in Section 14), so a user can move servers, or talk to users on other servers, without losing their name, graph, or history export. No module may assume all users live on the owner's server.
- **Keys are foundation, not a feature:** each user has device-held encryption keys, a recovery method the owner approves in Phase 0, and key verification for contacts. Private content is encrypted on the client before upload; a module that stores private content in plaintext on the server fails review, full stop.
- **AI permissions are foundation:** the AI section starts with access to NOTHING. The owner grants it per-module, revocable access (Reader, calendar, money view, messages, social), and every grant is visible in one settings screen. See Section 15.

The two-user test (definition of done for every module)

Maintain a second test account from week one. A module is done only when: User 2 *cannot* see User 1's private content, *can* accept an invite, *can* receive a message, and *can* open a document User 1 shared with them. A module that only works when one account exists in the world is an A-only hack and must be rejected in review.

7. Build Order

- **Phase 0 — Foundation:** multi-user auth/identity, profiles, contacts, invites, universal search shell, notifications, storage, audit log, visibility/permissions model. Passes the two-user test with empty modules.
- **Phase 1 — Solo-value modules:** the Reader (files/notes/docs, with the Listen button and the Speechify Library mirror as its first collection), calendar, email client, media player, photos, money view (Plaid), and the AI section in its basic form (chat + questions over the owner's permitted Reader/calendar/money data). Invites are live from launch. The app is the owner's daily driver while the network is still small.
- **Phase 2 — Network modules:** Messages (Snapchat-shaped, decluttered, federated + E2E), then in-app voice/video calls, then the one unified social module (Lemon8-style homepage; text/photo/video formats, one graph, federated), aimed at the inner circle (family, climbing team, reading group — real groups the owner already has). The AI section gains cross-module agent actions as each network module lands.
- **Phase 3 — Hardening for C:** moderation tooling, abuse/rate limits, scaling review, compliance basics — before any public launch, not after.
- **Phase 4 — Commerce (timing by partnership readiness):** wallet on partner rails, then native shop with own catalogue and Stripe checkout. Most regulated modules; deliberately last.

Rule for every phase and module: spec in plain English → build → owner runs it → tests pass (including the two-user test) → commit. Never start the next module on a broken previous one. Large multi-blocker failures in the owner's previous project (a decentralized raffle audited in October 2026) came precisely from compounding unchecked layers.

8. Claude Code Setup

MCP servers and plugins — install narrow, per job

Every enabled plugin/MCP server is a standing cost in context and a trust risk (plugins can execute code with the user's privileges; Anthropic does not security-audit MCP servers). Prefer official servers; use read-only database credentials where possible; uninstall what stops being used.

- **GitHub MCP** — PRs, issues, code search. Day one.
- **Supabase MCP (official) + Supabase Agent Skills** — tables, migrations, SQL, logs, edge functions; the skills carry Supabase's own guidance on schema design, row-level security, and auth. For this project's B-ready design, RLS correctness is the highest-stakes item on this list.
- **Context7** — current, version-specific library documentation injected on demand; prevents invented APIs.
- **Playwright MCP** — drives a real browser to verify built features; “it compiles” is not verification.
- **Chrome DevTools MCP** — console errors, network inspection, performance traces.
- **Native toolchain (v4):** Xcode + Swift Package Manager; a Swift/Xcode language-server or build MCP so the agent gets diagnostics after every edit, plus simulator and on-device runs as the verification loop. Playwright/Chrome DevTools remain for any web backend surfaces, not for app UI.
- **TypeScript LSP plugin** — only if TypeScript backend pieces (edge functions) are used; diagnostics after every edit and real code navigation.
- **Security Guidance + Code Review plugins** — structured review of generated diffs, mandatory for anything touching auth or user data.
- **Add later, per module:** Stripe MCP (commerce), Sentry MCP (after deploy), and deployment/signing tooling for iPhone + Mac builds. **Figma MCP/plugin is core, not optional** — the owner's design is in Figma; see Section 16. For federation work, add the Matrix / ActivityPub protocol documentation via Context7 before any protocol code is written.

Working configuration

- A tight **CLAUDE.md**: this brief's architecture, module rules, security non-negotiables, and the two-user test as standing instructions.
- **Custom subagents with separated roles:** builder, reviewer, test-writer. Code review happens in a different session/model context than building.
- **Hooks** that automatically run tests and linting after edits, so verification is not optional or forgettable.

GitHub — one repository from the outset (owner's requirement)

- **The repo exists before the first line of app code.** Phase 0 starts in GitHub, not in a local folder that gets uploaded later: the very first commits are this founding brief, the CLAUDE.md, and the design tokens (Section 16). History, decisions, and code grow together from day one — a repo created ‘once it works’ loses exactly the history that makes the project auditable and recoverable.
- **One private monorepo** under the owner's GitHub account, mirroring the modular monolith (Section 5): the SwiftUI app targets (iPhone + Mac), backend configuration, protocol/encryption code, docs, and design tokens in clearly separated module folders. One repo, not one repo per module — the module contracts in Section 6 live and are enforced inside it. Private while A/B; whether it ever goes public is a C-stage decision (Section 17).
- **Working discipline, even solo:** the main branch is protected; work lands through pull requests, because the PR is the review surface where the architect/auditor model (Section 9) and the Code

Review plugin do their work. Commits are small and conventional (one module-step per commit). GitHub Issues hold the OPEN decisions (Section 17) and each module's spec, so the plan and the code never drift apart. Tags mark each completed phase (Section 7).

- **CI from day one, at $0:** GitHub Actions (free tier, consistent with the A/B no-money rule) builds the iPhone and Mac targets and runs the test suite — including the two-user test (Section 6) — on every pull request. A red build blocks merging, for the AI builder exactly as for a human.
- **Secrets never enter the repo** — no API keys, tokens, certificates, or key material in code or history, ever (Section 11). Configuration lives in environment/GitHub Secrets. The repo must be safe to show an auditor; assume anything committed is permanent.

9. Model Strategy

- **Builder: Claude Fable in Claude Code.** Anthropic's published benchmarks put the Fable/Mythos class at the top on agentic coding (80.3% SWE-Bench Pro; 29.3% FrontierCode Diamond, per Anthropic's launch table as reported October 2026), with long multi-file tasks as its particular strength.
- **Mythos is not available:** same underlying model with safeguards lifted, restricted to vetted defenders through Project Glasswing. Public work uses Fable.
- **Architect/auditor: a different frontier model** (e.g. Muse Wonder via Muse Code, or the top GPT coding model) writes/reviews specifications and audits each module before the next begins. A model reviewing another model's code catches what self-review misses — a workflow the owner already uses (cross-evaluating models against each other).
- **Google Antigravity / Gemini frontier:** acceptable for UI iteration and visual variants; not the sole builder on a long project.
- Discipline rule: models build one module at a time against this frozen architecture. Free-styling module boundaries is how monoliths collapse.

10. Subscription Decision (owner's finances applied)

Current plan ladder (October 2026): Pro $20/mo; Max 5x $100/mo (5× Pro usage per session); Max 20x $200/mo (20× Pro). Chat and Claude Code share one usage pool. Fable is *not* included in Pro plan limits (usage credits only); on Max, Fable is included up to 50% of the weekly limit. Max also includes $100 (5x) / $200 (20x) per month in API credits for building on the Claude Platform — these do not cover Claude Code usage.

- **Decision: do NOT upgrade to $200/mo now.** The owner earns ~$16/hr at ~20 hrs/week (roughly $1,000–1,150/mo net), saves half of every paycheck, and has a binding savings target (a ~$3,400 study-abroad trip by March 2027, application due December 15, 2026). $200/mo is ~18% of take-home pay and would break his own savings rule.
- **Step 1:** build the foundation on the existing Pro plan. Use Opus/Sonnet for bulk work; buy Fable via usage credits only for the hardest modules (data model, messenger, security review).
- **Step 2 (upgrade trigger):** Max 5x ($100) only when Pro's session limits are hit mid-session repeatedly — roughly 3+ times a week for two consecutive weeks of real building. Evidence, not anticipation. Track usage with /usage in Claude Code.
- **Step 3:** Max 20x ($200) is for all-day professional use. Revisit only if the project becomes that, earns money, or the included $200 API credits become genuinely useful (e.g. powering this app's in-app AI

assistant).

11. Security and Legal Non-Negotiables

- **OAuth only.** Never store user passwords anywhere — including in this app. (The owner's own workflow models this: his Speechify password was provided only through a secure vault, never in chat.)
- Secrets in the backend secret store / environment only — never in app code, never in Git.
- Plaid access is read-only. Google scopes are minimal (e.g. Gmail read is not Gmail send unless a module truly needs it).
- One app holding identity + messages + banking tokens is a single point of total compromise. Threat-model it as such from Phase 0: RLS, audit logs, least-privilege credentials, and security review of every auth-touching diff.
- Payments and shopping launch only on licensed partner rails (Section 3, Class 3). No exceptions to “get users first” enthusiasm.
- **Encryption claims must be exact.** A feature may be labelled “end-to-end encrypted” in the UI only if the server genuinely cannot read it. Anything else is labelled honestly (e.g. “encrypted in transit”, “public post — signed, not private”). See the exception table in Section 14.
- **AI data rule:** context sent to the AI model is data leaving the E2E boundary unless inference is local/self-hosted. The app must show, per conversation, where inference runs and what module data was included. See Section 15.

12. Cost Posture

Stages A and B must cost nothing (owner's rule). The stack above is chosen so that is achievable: Supabase and other cloud services run on free tiers at A/B scale, and the long-term hosting direction is the owner's own **Raspberry Pi home server** — electricity he already pays for — for the backend, federation homeserver, media, and possibly local AI inference. If a feature cannot be done on a free tier or the Pi, it waits for C or gets redesigned; it does not quietly start a subscription.

- **Known friction — iPhone distribution:** Apple's free provisioning lets the owner install his own build on his own iPhone at $0, but free-provisioned apps expire after 7 days and must be re-installed/re-signed. The Apple Developer Program ($99/year) removes that and enables TestFlight for the inner circle (B). Under the A/B no-money rule the default is free provisioning; whether the $99 is worth it is an OPEN decision for the owner (Section 17) — Claude Code must not assume it.

13. Social and Messages — Design Specifics

Social: one unified module, Lemon8-style homepage

- **One post model:** a Post has a body (text), zero or more media attachments (photos and/or a vertical video), and a format derived from its attachments. Do not create separate tables/apps for text posts, photo posts, and videos.
- **One graph:** follows, blocks, mutes, likes, reposts, bookmarks/saves, and comments are format-agnostic — the same system serves a text post and a video.
- **One profile:** a user's profile shows all their formats in one grid/timeline with format filters, not three separate accounts or tabs-as-apps.

- **Homepage (owner's decision — Lemon8 is the model):** the opening screen is a discovery feed, not a bare chronological timeline: a two-column masonry of cover cards, topic/category chips and search prominent at the top, and cards designed to look good for every format — photo posts show the photo, video posts show a thumbnail with duration/play badge, text posts show a typographic cover card (title/first lines on the design system's card treatment). Tapping a card opens the full post in its native reading/viewing layout. A Following view stays one tap away; any algorithmic ranking beyond the discovery homepage's ordering is a later, explicit decision.
- **Composer:** one composer; the owner picks (or the attachments imply) text, photo, or video. Lemon8-style touches worth copying: cover/title step for every post, topic tagging at publish time, and simple built-in photo templates/filters rather than a full editor.
- **Video pipeline:** short video is the expensive format (storage, transcoding, CDN). Use a managed pipeline (e.g. Mux or Cloudflare Stream) when this module is built; do not hand-roll transcoding. Note the Section 14 tension: a managed public-video pipeline processes public posts only — private media stays user-key encrypted.
- **Figma note:** the owner will send a bare-bones Figma design for this homepage (and the app generally); when it arrives it overrides these words where they conflict.

Messages: Snapchat-shaped, decluttered

- **Home screen:** one clean list — friends/conversations with snap/chat preview, unread state, and a camera button. That is the whole first screen. Search friends at the top; nothing else competes for space.
- **What is kept from Snapchat:** visual-first messaging (send a photo/video snap as easily as a text), camera one swipe or one tap from the chat list, close-friends list, stories only from friends (if stories are built at all — they belong to Messages here, not a second social surface), optional per-chat disappearing messages, and quick reactions.
- **What is cut (the clutter):** no Discover, no Spotlight, no public-content tabs, no ads, no sponsored stories, no Snap Map by default (location sharing is explicit, per-friend, per-occasion only), no engagement-bait streak pressure. Maximum three tabs: Chats, Camera (or Stories), Profile/Settings.
- **Under the UX:** messages ride the federated E2E protocol in Section 14 — Snapchat's shape, none of its closed-network or server-readable properties.

Groups: Discord-like communities

- **Communities/spaces:** a group is a named community (e.g. the climbing team, the reading group) containing multiple topic channels — announcements, general, media, planning — rather than one undifferentiated group chat. Channels are text-first; voice channels are a later add-on on the Section 3 calling stack.
- **Threads and structure:** replies can branch into threads inside a channel; channels can be organised in simple categories; light roles (owner / moderator / member) control who can post in announcement-style channels. Keep Discord's structure, skip its bloat: no server boosting, no app directory, no public discovery in A/B.
- **On the protocol:** Matrix rooms/spaces map naturally onto communities/channels and stay federated + E2E — one more reason Matrix is the default messaging candidate in Section 14. Group size in B is small (his real groups), so group-E2E limits are not a practical constraint at this stage.

14. Federation and End-to-End Encryption — Locked Principle

Owner's decision, verbatim in substance: **everything federated, everything end-to-end encrypted.** Claude Code implements it as follows — as a default with named, honest exceptions, because a literal reading is technically impossible for three categories and the brief does not authorise lying about encryption in the UI.

- **Federated means:** open protocols, portable identity (@user@server), the ability to follow/message users whose accounts live on other servers, server choice (the owner runs the first server; B users may join it or their own), and full data export. Protocols: **Matrix** for messages/calls signalling; **ActivityPub and/or AT Protocol** for social — the Phase 0 proposal must pick, with reasons (ActivityPub = widest existing federation, Mastodon-compatible; AT Protocol = Bluesky's model, stronger identity portability, younger social ecosystem). Email and calendar already federate by nature (open protocols) — keep them that way, no proprietary lock-in wrapper.
- **End-to-end encrypted means:** private content is encrypted on the sender's device and readable only by its intended recipients' devices. Messages and 1:1 calls: E2E always, no plaintext mode. Private/shared files, docs, notes, photos, and AI memory: encrypted with user keys before upload; the server holds ciphertext. Group calls and large group chats use the protocol's group E2E (MLS/Megolm-class) with its limits stated in the Phase 0 proposal.
- **Exception 1 — public social posts:** a public, federated post cannot also be secret; federation means other servers receive it. Public posts are **signed** (authentic, tamper-evident) and travel over encrypted transport, but are not E2E-private. Followers-only and private posts, by contrast, ARE distributed only to intended recipients and encrypted accordingly where the chosen protocol supports it — the Phase 0 proposal must state exactly which visibility levels get true E2E on the chosen protocol.
- **Exception 2 — ordinary email and external feeds:** Gmail/IMAP email, RSS news, and Plaid bank data arrive from systems that are not E2E. Store them user-key encrypted at rest inside the app where feasible, label them honestly, and never claim E2E for mail sent to outside addresses (an optional PGP-style mode is out of v1 scope).
- **Exception 3 — regulated rails:** payments and PSTN phone calls pass through licensed providers by law and design; they are encrypted in transit, never marketed as E2E.
- **Costs of this principle, accepted by the owner and to be planned for:** server-side search over private content is impossible (universal search over private modules happens on-device / over local indexes); password-style account recovery cannot restore E2E history without a key backup (Phase 0 must propose the recovery-key design); moderation over E2E content is report-based, not server-scanning; multi-device sync is key-sync work, not a free feature.
- **Supabase's role shrinks accordingly:** it may hold accounts, public data, and ciphertext, and solo-module data the owner explicitly chooses to keep server-readable (if any) — each such choice is listed in the Phase 0 proposal and approved line by line.

15. The AI Section — Open-Source Assistant (Qwen)

Owner's decision: a dedicated **AI section** in the app, powered by an **open-source / open-weights model — Qwen in the first instance** — that **acts like Muse**: not a generic chatbot tab, but his personal assistant inside his own app, with the same job Muse does for him today.

- **Model layer:** Qwen-family open weights by default, served behind an OpenAI-compatible inference interface (self-hosted with a vLLM/Ollama-class stack, or a hosted open-weights provider the owner

chooses). The model is a replaceable setting (Qwen size/variant, or another open family later) — no module may hard-code a vendor. Small/local variants are the on-device or home-server option for private work; larger hosted variants are opt-in per conversation.

- **What ‘acts like Muse’ means, concretely:** (1) a persistent, user-owned **memory** — facts, preferences, people, and ongoing threads the assistant keeps across conversations, visible and editable by the owner in the app, stored user-key encrypted; (2) **cross-module tool use** under the Section 6 permission grants — search and read Reader files (including the Speechify Library mirror), summarise documents and start Listen playback, check calendar, draft (not silently send) messages and emails, surface a money snapshot, create notes/docs/tasks; (3) **agent behaviour with approvals:** multi-step tasks are proposed, executed stepwise, and anything outward-facing (sending, posting, paying, deleting) stops for the owner's explicit approval, exactly as Muse does; (4) conversation history and proactive check-ins/summaries live in the AI section, not scattered across modules.
- **Honest capability note:** an open-weights Qwen model will not match a frontier hosted assistant (Muse) on the hardest reasoning, coding, or long-context work, and the app must not pretend otherwise. Design the section so the hard jobs can be routed — with the owner's explicit choice — to a stronger hosted model later, while the default, private, always-available assistant is his own model on his own terms.
- **Privacy rule (ties to Section 14):** the AI section shows where inference runs (local/self-hosted vs. named hosted provider) and what module data was included in each conversation. With local/self-hosted inference, permitted module data can stay inside the owner's boundary; with hosted inference, included data leaves the E2E boundary and the UI says so before it happens, not after.

16. Design System, UI/UX Resources, and Reference Repositories

Owner's requirement: **this must not look like vibe-coded slop.** The known failure mode has a name in current practice: the “shadcn trap” — an unmodified shadcn/ui install (slate/zinc palette, Inter font, default radius) makes every AI-built app competent and forgettable, because agents reach for the most common pattern. The fix is not a different library; it is making the defaults *yours* before any module is built.

Anti-slop rules (standing instructions)

- **Design tokens first, before features:** set the colour palette (locked below), ONE saturated accent colour used sparingly, border radius, type scale and font pairing, and spacing scale as named design tokens (SwiftUI constants + a design.md token file) that Claude Code must read every session. Changing roughly five core tokens is what separates a branded product from the default template.
- **Visual style and colour scheme (owner's decision): NEUMORPHIC — white and dark grey.** Surfaces use soft, extruded neumorphism: light mode is built on white / very light grey surfaces with soft dual shadows (light highlight + soft grey shadow) so controls look gently pressed or raised from the background; dark mode mirrors it in dark greys (charcoal surfaces, subtler shadows). Background and surface colours stay close in value — that closeness IS the neumorphic effect — with content contrast carried by typography weight, the single accent colour, and imagery (feed cover cards, photos). Rules: no rainbow palette, no pure-black backgrounds, no harsh drop shadows or hard borders as the default treatment; elevation and grouping come from the soft shadow pair, not lines. Accessibility guardrail: body text still meets contrast minimums (use the darker greys for text on light surfaces), because neumorphism done naively fails readability — the look never wins over legibility.

- **Compose, don't invent:** agents are far more reliable at composing a finite, curated component vocabulary than at hand-writing CSS from scratch. All UI is assembled from the design system's components; bespoke one-off styling is a review failure.
- **Figma is the visual source of truth.** The owner is producing a bare-bones Figma design, which will be added to this brief as an appendix when supplied. Once it lands, it overrides any default styling: tokens, layout, and component shapes are extracted from the Figma file, not invented. Use the **Figma MCP (Dev Mode MCP server) / Figma plugin for Claude Code** so the coding agent can read the Figma design directly instead of working from screenshots or descriptions.
- **Motion with a system:** one motion library (Motion / Framer Motion), a small set of named durations and easings used everywhere. Animated component registries (below) supply at most a few signature pieces in product UI — they are marketing-shaped and thin on tables, forms, settings, and empty states; a product is mostly those unglamorous components, which come from shadcn/Origin-style registries.
- **Icons:** Lucide only, one stroke weight. No emoji as icons. **Content:** real content in every mock and test — never lorem ipsum; slop hides in fake content.
- **Visual verification loop:** every module is screenshotted via Playwright, compared against the Figma/tokens, and reviewed before commit — typography, spacing rhythm, and empty/loading/error states are explicitly checked, because default-generated UIs skip all three.

UI/UX libraries and integrations to use

Resource	Role in this project
shadcn/ui (on Radix UI)	Component substrate — source code lives in the repo, so the agent can read and edit it directly. Always with custom tokens (see the trap above)
tweakcn	Token/theme editor for shadcn — the fast way to set the custom palette, radius, and fonts
Origin UI	Product-grade shadcn-compatible components (dialogs, calendars, commands, forms) — the unglamorous components the app actually runs on
Magic UI / Aceternity UI / React Bits	Animated components — use sparingly: one or two signature pieces, never as the design system
21st.dev (+ its Magic MCP for coding agents)	Searchable community component registry an agent can pull from — for prototyping and one-off pieces, with a consistency cleanup pass afterwards
Motion (Framer Motion)	The one animation library — layout transitions, feed and player micro-interactions
Lucide	The one icon set
Figma Dev Mode MCP + Figma plugin (Claude Code)	Lets Claude Code read the owner's Figma design directly — the bridge between his design and the code

Home screen — Apple Watch-style bubble design (owner's decision)

- **The app's home page is modelled on the Apple Watch app launcher:** a fluid cloud of circular bubbles — one bubble per module — arranged in a honeycomb that extends beyond the screen edges. The owner pans/drag-drifts through the cloud; bubbles nearer the centre render larger, peripheral ones smaller, giving the watchOS sense of a space you move through rather than a grid you scroll. Tap a

bubble to open its module.

- **Rendered in the locked neumorphic scheme:** each bubble is a soft white / dark-grey neumorphic disc (Section 16) with the module's Lucide icon centred; the module the owner uses most can sit larger at the centre by default. Arrangement is owner-editable (long-press to rearrange, watch-style), and ordering defaults to frequency of use.
- **Glanceability, used sparingly:** a bubble may carry a small badge or inner indicator — unread message count, a now-playing pulse on the player/Reader bubble, the next calendar event's time — so the home screen is informative without turning into a dashboard. Which glance data each bubble shows is the owner's later detail call (Section 17).
- **Honest constraints:** the bubble cloud is a launcher, not a content surface — content lives inside modules (the social module keeps its own Lemon8 homepage). Provide an accessibility/list alternative view of the same modules (bubbles are delightful but slower for some users and some tasks), and on the Mac the home screen adapts: the bubble cloud as the launch view, with a conventional sidebar once inside a module.

Owner-flagged resources (noted Oct 28, 2026, at the owner's direction)

- **github.com/topics/reverse-engineer-anything** — GitHub topic index of reverse-engineering projects: the reference shelf when a module needs to understand how an existing app implements a behaviour he uses.
- **github.com/acvnace/awesome-vibe-coding-resources** — a curated awesome-list of vibe-coding resources: tools, MCP servers, prompts, and workflows for AI-assisted building.
- **githublb.vercel.app/topic/vibe-coding** — a browsable topic index of vibe-coding repositories, for finding prior art and reusable pieces by topic.
- **reddit.com/r/ClaudeAI — "How to use Claude Code, presented by Anthropic"** (thread 1ktic8h) — Anthropic's own Claude Code guidance as discussed on r/ClaudeAI; the workflow reference for how the builder agent should be driven on this project.

Reference / recreation repositories (study and pattern-mine; respect each licence)

These are real open-source products and clone collections. Their job is reference: how a feed, a player, a photo library, or a chat client is actually structured and styled — read them, extract patterns, do not blindly copy code (check each repo's licence first, and never copy branding/assets).

Repository	What it is the reference for
bluesky-social/social-app (+ atproto)	THE reference for the unified social module: one post record carrying text, image, or video embeds; feeds; DMs; moderation — a production social app, actively maintained
mastodon/mastodon	Second social reference: federated timelines, moderation tooling at real scale
GorvGoyl/Clone-Wars (GitHub)	Curated index of 100+ open-source clones — Instagram, TikTok, WhatsApp, Spotify, Amazon, YouTube and more, with demos and stacks; the lookup table when a specific feature (video feed, music player, checkout) needs a proven implementation to study
immich-app/immich	Photos module reference: a polished Google Photos alternative — upload pipeline, thumbnails, albums, timeline UX

Repository	What it is the reference for
AppFlowy-IO/AppFlowy	Reader reference: a Notion-style workspace — editor, databases, page tree UX, for the Reader's documents and notes
element-hq/element-web	Messaging reference: a production Matrix chat client — rooms, threads, encryption UX
livekit/livekit (+ its examples)	Calling reference: production WebRTC infrastructure and sample call UIs
actualbudget/actual	Finance reference: local-first personal finance UX — budgeting and transaction views to learn from for the money module
medusajs/medusa (or Saleor)	Commerce reference for Phase 4: headless catalogue/checkout architecture when the shop is built
Lemon8 (product, not a repo) + Snapchat (product, not a repo)	Design references only: Lemon8 for the social homepage (masonry discovery feed, topic chips, cover cards); Snapchat for Messages' shape (camera-fast visual chat, friends list) MINUS its clutter tabs. Study the products; copy no branding or assets
QwenLM/Qwen (open-weights models + serving docs)	AI section reference: the default open model family, its sizes/variants, tool-use and context capabilities, and self-hosting requirements (Section 15)

17. OPEN Decisions (ask the owner; do not guess)

- **Identity basis — DECIDED (Oct 28, 2026):** handle + email, phone optional. The handle is the public identity and the email is the account credential/recovery anchor; a phone number may be added by the user but is never the identity and never required. Everything keys off this.
- **Product name — DECIDED (Oct 28, 2026): Totality.** Chosen by the owner over ‘World’, which collides with the existing World App (World, ex-Worldcoin). The name states the project's point in the Hegelian sense: the whole that exists only through its parts. A stage-C trademark search is still required before any public launch.
- **Home screen:** DECIDED in form — an Apple Watch-style bubble cloud of module bubbles (Section 16). Still open in detail: which glance indicators each bubble carries (unread counts, now-playing, next event, money snapshot) and the owner's preferred default arrangement/centre bubble.
- **Figma design (PENDING from owner):** the owner is producing a bare-bones Figma design — now including the Lemon8-style social homepage and the decluttered Snapchat-style Messages screens — and will send it; when it arrives it is added to this brief as an appendix and becomes the visual source of truth per Section 16. Until then, design tokens are provisional placeholders, not final brand decisions.
- **Social federation protocol:** ActivityPub vs. AT Protocol (Section 14). The federation + E2E principle itself is LOCKED and is not open; only the protocol pick is, and the Phase 0 proposal must recommend one with reasons.
- **E2E key recovery design:** recovery-key / trusted-device scheme for regaining E2E history on a new device. Phase 0 proposes; owner approves.
- **AI inference hosting:** self-hosted Qwen (owner's hardware / rented GPU) vs. hosted open-weights provider vs. both (local default, hosted opt-in). Affects cost and the Section 15 privacy posture.
- **Real phone calling:** in scope for a later phase via a PSTN provider, or permanently out? Affects identity and cost model.

- **iPhone distribution:** free provisioning ($0, apps expire after 7 days) vs. Apple Developer Program ($99/year, TestFlight for B users). The A/B no-money rule defaults to free provisioning — the owner decides if/when the $99 is justified (Section 12).
- **iPhone Duo specifics:** exact form factor, sizes, and hinge/fold behaviour when the device ships; the adaptive-layout requirement (Section 4) is locked, the concrete Duo layouts are designed when hardware details exist.
- **Raspberry Pi timing and spec:** when the Pi server is set up, which Pi/storage, and which services migrate first (backend vs. federation homeserver vs. AI inference). A/B must not depend on the Pi until it exists.

18. First Task for Claude Code

Propose — as a reviewable document, before any feature code — (1) the Phase 0 architecture: data model with owner_id and visibility on every entity, the RLS policy sketch, the federated identity/contacts/invite design, the federation protocol pick and E2E/key-recovery design (Section 14), the AI inference and permission design (Section 15), and the module-boundary contracts; and (2) a CLAUDE.md encoding this brief's rules. The owner and an independent model review that proposal against Sections 2, 6, 11, 14, and 15 of this brief. Only after approval does module building begin, in the order of Section 7. Step zero, before even the proposal is final: create the GitHub repository (Section 8) — this brief and the CLAUDE.md are its first commits.

End of brief — v4, October 28, 2026. Compiled from the owner's design discussion of October 28, 2026: essentials inventory, native-integration clarification, A→B→C scope decision, unified social module, Reader-with-Listen, Lemon8 social homepage, decluttered Snapchat-style Messages, federation + E2E principle, Qwen AI section, Claude Code tooling, subscription analysis — plus the v4 revisions: native iPhone app first (not a web app), a connected Mac app, eventual iPhone Duo support, Discord-like group messaging, the A/B no-money rule, Amazon-LIKE (not Amazon-wrapper) shopping, Raspberry Pi self-hosting as the hosting direction, a neumorphic white/dark-grey colour scheme (Section 16), and the owner's four flagged resource links (Section 16).